Cyberbullying Detection

Mattia Segreto

1 Introduction

In last years, the phenomenon of cyberbullying increases significantly; this is likely due to the social network global spread and the growing accessibility of communication tools. To recognize and counter harmful behavior is an urgent challenge in order for promoting a safer digital environment.

The project aim is to propose a detection system for cyberbullying attacks based on the analysis of tweets. Following the detection, a second purpose has been identified: to provide an explanation of how the system reached such a conclusion.

2 Methodology

This chapter presents the methodological framework adopted for the development of the cyberbullying detection system. The approach follows a structured pipeline that includes data preprocessing, feature extraction, supervised learning techniques, and model evaluation. Particular attention is given to the classification process, which is designed in multiple stages to enhance accuracy and interpretability. Furthermore, explainability methods are integrated to improve transparency and align the system with current ethical and regulatory standards.

2.0.1 Data Visualization

Tag clouds were generated for each of the six dataset classes in order to provide a visualization that provides a qualitative overview of the most frequent words associated with each label and offering also an initial insights of the characterizing language patterns among different categories.

Figure 1: Tag clouds for each class.

The tweet lengths distribution across classes was also take in consideration through a boxplot, in order to assess possible differences in verbosity among classes. No significant patterns were identified from this analysis. Consequently, no features related to tweet length were created and included in the final model.

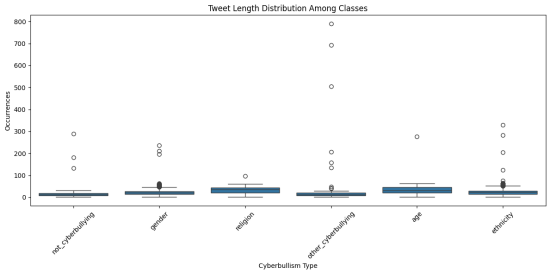


Figure 2: Distribution of tweet lengths per class.

2.0.2 Data Splitting and Stratification

The dataset was split into training and test sets using an 80/20 ratio. Stratified sampling was applied based on the multiclass label in order to preserve the original class distribution in both subsets. A fixed random seed was used to ensure reproducibility of the split.

2.1 Data Collection and Preprocessing

2.1.1 Dataset Description

The following dataset description is based on the dataset introduced in the paper *SOSNet: A Graph Convolutional Network Approach to Fine-Grained Cyberbullying Detection*, which was specifically created to support fine-grained classification of cyberbullying on Twitter. This dataset contains almost 45000 tweets labelled according to the class of cyberbullying:

- Age;

- Ethnicity;
- Gender;
- Religion;
- Other type of cyberbullying;
- Not cyberbullying

It has two columns that are:

- tweet_text: the raw content of the tweet (string format).
- cyberbullying_type: the corresponding label, which can be either not_cyberbullying or one of several cyberbullying types.

The dataset used in the SOSNet model was initially built by merging six public datasets focused on cyberbullying on Twitter, such as those by Waseem, Davidson, and Chatzakou. Since many of these datasets contained only tweet IDs, the authors retrieved the corresponding tweet texts using the Twitter API, though only a portion could be recovered due to tweet deletions over time. The resulting dataset was highly *imbalanced*, with a strong dominance of the *gender* and *other* classes (about 85% of the total), while the *age* and *ethnicity* classes were severely underrepresented (less than 2.5%). To address this issue, the authors introduced a *Dynamic Query Expansion (DQE)* [1]; an iterative process that allows the authors to naturally and effectively balance the dataset.

2.1.2 Text Cleaning

In order to reduce noise and standardize the input for the feature extraction step, a set of preprocessing techniques was applied to the raw tweets. These steps are especially important in the context of Twitter data, which is typically informal, highly variable, and often includes non-linguistic symbols. The cleaning procedure includes:

- **Lowercasing:** All text is converted to lowercase to ensure that words like `Hate` and `hate` are treated as the same token. This helps reduce the vocabulary size without losing semantic meaning.
- **Removal of URLs, mentions, and punctuation:** Tweets often contain hyperlinks, user mentions (e.g., `@username`), and various punctuation marks that do not contribute meaningfully to cyberbullying detection. Removing them simplifies the input without affecting classification performance.

[1] DQE is a semi-supervised strategy that does not generate synthetic tweets, but instead retrieves real tweets from Twitter. It starts from manually selected *seed queries* (e.g., "middle school" for the *Age* class). These queries are used to collect new, real tweets through the `GetOldTweets3` library. The process is repeated iteratively, updating the keywords each time, until a sufficiently rich and balanced dataset is obtained.

- **Stopword removal:** Common words such as `the`, `is`, and `and` are removed to focus on the most informative terms in each tweet. This is particularly helpful for short texts like tweets, where maximizing the signal-to-noise ratio is essential.
- **Stemming:** Words are reduced to their root form (e.g., `harassing` → `harass`), which helps group different inflected forms of the same word under a single representation. This reduces sparsity and improves the effectiveness of models relying on token frequency or similarity. Lemmatization was not used because, given the unstructured nature of the text, it would have been less efficient: lemmatization relies on correct grammatical structure and part-of-speech tagging, which are often unreliable in noisy or informal text.

These choices make the text cleaner, more compact, and more suitable for feature extraction methods, while preserving the linguistic signals needed for cyberbullying detection.

Binary Label Creation: To support the two-stage classification pipeline, the original multiclass label has been accompanied by a binary label distinguishing between cyberbullying and non-cyberbullying content. In this transformation, all tweets originally labeled as `not_cyberbullying` were grouped under the class `0` (non-cyberbullying), while all remaining tweets were grouped under the class `1` (cyberbullying). This binary representation is particularly useful as a preliminary filter in a cascaded classification system, where the first stage detects whether a tweet is abusive at all, and only the second stage performs fine-grained categorization among the different types of cyberbullying. Such a design reduces the complexity of the fine-grained classifier and helps focus its training on content that is more likely to be harmful.

2.2 Rebalancing

The dataset distribution can be seen through this plot:

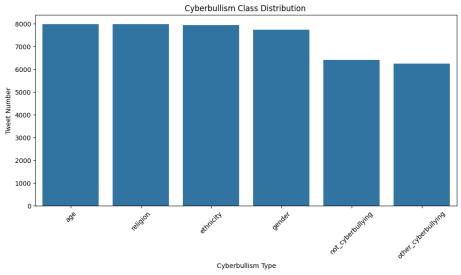


Figure 3: dataset classes distribution

It is evident that in the multiclass scenario, the labels are overall quite balanced, with the only exception being the `other_cyberbullying` class, which is slightly underrepresented. However, in the binary classification setting, the dataset is significantly imbalanced, making it necessary to adopt a rebalancing technique suitable for the project's objectives. Choosing an effective class rebalancing strategy proved to be far from straightforward. Initially, SMOTE (Synthetic Minority Over-sampling Technique) was considered, as it is commonly used to generate synthetic examples for the minority class. However, SMOTE relies on a continuous feature space, whereas the Bag of Words (BoW) representation, which is one of the vectorization techniques that will be used, uses discrete vectors based on integer frequencies. Even with rounding strategies applied, the use of SMOTE led to unsatisfactory results, generating term combinations that lacked both semantic and syntactic coherence, thereby undermining the linguistic validity of the synthetic data. Similarly, standard techniques such as undersampling and oversampling were deemed unsuitable. Undersampling would have resulted in a substantial loss of information by discarding many majority class examples, while oversampling would have introduced high redundancy, increasing the risk of overfitting on the minority class. Given the limitations of general rebalancing strategies, attention shifted to data augmentation methods specifically tailored for the text mining domain. Among the considered approaches, synonym replacement and back translation emerged as the most promising in terms of computational feasibility and alignment with the dataset's characteristics. Back translation, although effective for well-structured texts, proved less suitable in this context due to the informal, syntactically irregular, and often abbreviated nature of tweets. As a result, synonym replacement was ultimately chosen, as it introduces lexical variability without altering syntactic structure, thereby preserving both semantic coherence and natural expression. Finally, for the sake of simplicity and methodological consistency, the same approach was applied to the multiclass case, despite the absence of critical imbalances as encountered in the binary scenario.

2.3 Feature Extraction

In this project, three different text vectorization methods were employed to extract meaningful features from the tweets: Bag of Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), and Word2Vec(W2V-1) embeddings. These techniques allow the conversion of raw textual data into numerical representations suitable for downstream classification tasks.

BoW was used to construct a frequency matrix that reflects how often each word appears across the dataset. While effective in capturing general word usage, this method tends to overrepresent common terms such as *like*, *people*, or *know*, which may carry limited discriminative power for identifying cyberbullying content.

To mitigate this, the TF-IDF approach was also applied. It enhances the BoW representation by down-weighting ubiquitous terms and up-weighting words that are more unique to specific tweets. This helps focus the model on more

informative and discriminative tokens, particularly useful in a dataset where repeated expressions of emotion or aggression are relevant.

Additionally, a Word2Vec model was trained to capture semantic and syntactic relationships between words. The model was trained on a large, unlabeled corpus composed of two datasets related to cyberbullying and offensive content, totaling approximately 120,000 preprocessed sentences. The Skip-Gram architecture (`sg=1`) was used, with a `vector_size` of 200 and a `window` of 7. Different `min_count` values were evaluated: setting it to 2 excluded 50,868 words out of 69,392 (73.3%), retaining 18,524 tokens. A lower threshold (1) excluded 62.4%, while values of 5 or 10 would have eliminated over 83% and 88% of the vocabulary, respectively.

This multi-faceted feature extraction setup provided several distinct vector representations of the text, allowing models to be trained and compared across different input spaces.

2.4 Classification Overview

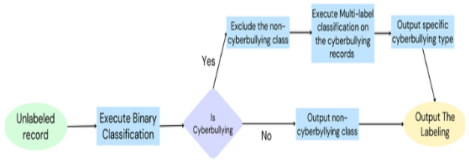


Figure 4: Two-stage classification pipeline.

Source: Self-Training for Cyberbully Detection: Achieving High Accuracy with a Balanced Multi-Class Dataset, available at `https://uregina.ca/~nss373/papers/cyberbully-detection.pdf`

The two-stage classification improves the system's interpretability: by splitting the process into detection and specific categorization, it becomes easier to identify where an error occurs and intervene accordingly, with greater precision in tuning and feature analysis.

Moreover, this structure mirrors human reasoning, which typically involves first assessing whether content is problematic and only then analyzing its severity or nature. As a result, errors can also be interpreted on two different levels of severity: failing to detect harmful content is clearly more critical than misclassifying the specific type of cyberbullying. This is especially true since some offensive messages may not fit neatly into a single label, and therefore an error at the multiclass stage does not necessarily represent a serious failure. This approach enables a more realistic understanding of the model's behavior and a more informed management of its performance in practical applications. For example, consider the following case: the ground truth label for the tweet "my 2282 says that you can replace the testimony of one man with that of two women" was set as religion, whereas the model predicted gender—and arguably,

it was correct to do so.[2] This suggests that the reported results may underestimate the models' actual real-world performance, as some supposed "errors" stem from questionable ground truth labels rather than true misclassifications.

2.4.1 Binary Classification Stage

The binary classification stage aims to distinguish between tweets containing cyberbullying and those that are harmless. Each tweet is labeled as either `cyberbullying` or `not_cyberbullying`, framing the task as a supervised binary classification problem.

Nested cross-validation was performed for each of the nine model–vectorizer combination pipelines to obtain unbiased performance estimates and tune hyperparameters.The classifiers that has been used are:Random Forest, Logistic Regression, and Linear SVM. Model evaluation employed a nested stratified k-fold framework: an outer loop estimated generalization performance, while an inner loop conducted hyperparameter tuning. From each inner fold, the optimal parameter set was recorded, and upon completion of all folds the final hyperparameters were chosen as the most frequent combination ("mode") across folds. That configuration was then used to retrain the pipeline on the entire training set. Optimization targeted the F1 score to balance precision and recall, thus addressing both false positives and false negatives. Model selection involved pairwise comparisons among all pipelines, with a "win" noted when one model outperformed another. To ensure robustness and ample sample size, all comparisons were based on a repeated stratified k-fold cross-validation scheme. The normality of per-fold F1 differences was assessed using the Shapiro–Wilk test; if normality held, a paired t-test was applied, otherwise the Wilcoxon signed-rank test was used.

2.4.2 Multiclass Classification Stage

The goal of the multiclass classification stage is to assign a specific type of cyberbullying to each tweet previously identified as harmful. The hyperparameter-tuning procedure was identical to that used in the binary stage(without rebalancing step): each of the three classifiers (Random Forest, Logistic Regression, and Linear SVM) was paired with each of the three vectorization methods, yielding nine model configurations. Nested stratified k-fold cross-validation was employed to optimize hyperparameters;this time targeting macro-f1 and, once tuning was complete, each pipeline was retrained on the full training set. Model selection again relied on pairwise comparisons of per-fold scores (wins counted per fold) with significance assessed via Shapiro–Wilk for normality and paired t-tests or Wilcoxon signed-rank tests as appropriate.

[2]The tweet likely refers to verse 2:282 of the Quran, which discusses financial contracts and stipulates that, if two male witnesses are not available, one man and two women may testify. Although the rule originates from a religious text, the content of the tweet explicitly highlights a gender-related issue, leading the model to a plausible alternative classification.

2.5 Explainability Module

To improve model transparency and interpretability, an explainability module was integrated into the classification pipeline. This component plays a crucial role in enhancing trust and accountability in machine learning applications, especially in sensitive tasks such as cyberbullying detection. Both local and global explanability were provided.

Global Explainability. Pattern mining techniques were applied to study the most frequent and informative word associations within the dataset. Specifically, the most frequently occurring words were analyzed, and maximal and closed itemsets meeting a predefined support threshold were extracted. Rather than focusing on association rules, which are better suited to modeling directional relationships between items, maximal and closed itemsets has been chosen. This choice was motivated by the nature of the data: tweets are extremely short and informal, making it difficult to establish strong directional associations between words. Maximal and closed itemsets allowed us to capture robust co-occurrence patterns without introducing noise or spurious dependencies, providing a clearer and more stable view of the most informative word groupings associated with different cyberbullying categories. In parallel, directly on classifiers, the *feature_importance* attribute of the Random Forest model (which showed better performance) was used to rank features by their global relevance. This dual approach enables a comprehensive form of global explainability: on one hand, it provides an understanding of the model's behavior through feature importance; on the other, it offers an intrinsic explanation of the phenomenon itself by uncovering the underlying word patterns through pattern mining.

Local Explainability. For multiclass instance-level explanations, the TreeInterpreter library was employed. These method decomposes the output of the models into individual feature contributions, making it possible to understand which specific words influenced the classification of a given tweet. Thanks to the use of TF-IDF vectorization (that is the vectorizations used which best performs in multiclass and binary classifiers), each explanation could be interpreted in terms of the positive or negative contribution of each word. Interestingly, the model also relies on the absence of certain words: even terms not present in the tweet can carry weight by shifting the decision boundary. This aspect highlights the nuanced reasoning adopted by the classifier and provides users with a deeper understanding of its behavior.

3 Experimental Results

This section presents the results obtained at each stage of the proposed pipeline, following the methodological steps described previously. For each stage, the evaluation metrics used, the performance of the tested models, and a short discussion of the results were reported.

3.1 Binary Classification Results

Experimental Setup and Grid Search. As described in Section 2.4.1, three classifiers Random Forest, Logistic Regression, and Linear SVM were evaluated in combination with three vectorization methods: Bag of Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), and custom-trained Word2Vec embedding (W2V-1). This led to a total of 9 distinct model configurations. The grid of hyperparameters used during tuning was the following:

- **Logistic Regression:**
  - `C`: {0.01, 0.1, 1}
- **Linear SVM:**
  - `C`: {0.01, 1, 10}
- **Random Forest:**
  - `n_estimators`: {100, 500}
  - `max_depth`: {None, 20}
  - `random_state`: 42

Model evaluation Table 1 reports the optimal hyperparameters that were selected via nested cross-validation for each vectorizer–classifier pipeline. For each configuration, it's been showed the mode of the best parameters found across the ten outer folds.

Configuration	Chosen Parameters
BoW + LogisticRegression	{'C': 1, }
BoW + RandomForest	{'max_depth': none, 'n_estimators': 500, 'random_state': 42}
BoW + LinearSVM	{'C': 10}
TF-IDF + LogisticRegression	{'C': 1}
TF-IDF + RandomForest	{'max_depth': none, 'n_estimators': 500, 'random_state': 42}
TF-IDF + LinearSVM	{'C': 1}
W2V-1 + LogisticRegression	{'C': 0.01}
W2V-1 + RandomForest	{'max_depth': 20, 'n_estimators': 500, 'random_state': 42}
W2V-1 + LinearSVM	{'C': 0.01}

Table 1: Chosen parameters for each configuration (mode over 10 folds)

Table 2 summarizes the key evaluation metrics: mean accuracy, precision, recall, F1 score and F1_macro score along with their standard deviations across the outer folds. This allows you to compare not only the average performance but also the stability of each model.

Vectorizer	Classifier	Mean f1	Mean Acc.	Mean Prec.	Mean Rec.	Mean F1_macro	Std f1_macro	Std Acc.	Std Prec.	Std Rec.	Std F1
TF-IDF	RandomForest	0.905981	0.848335	0.964091	0.854501	0.756927	0.006841	0.004786	0.003522	0.005352	0.003140
BoW	RandomForest	0.903343	0.844219	0.962246	0.851271	0.751002	0.006680	0.004689	0.003934	0.005961	0.003133
W2V-1	RandomForest	0.902507	0.842162	0.956441	0.854370	0.744036	0.007847	0.005993	0.003713	0.007119	0.004008
TF-IDF	LogisticRegression	0.886757	0.821357	0.968221	0.817979	0.731874	0.005023	0.005074	0.002687	0.007111	0.003679
TF-IDF	LinearSVM	0.883932	0.817833	0.971030	0.811221	0.730305	0.004621	0.004873	0.003199	0.007330	0.003621
BoW	LogisticRegression	0.881775	0.815494	0.975295	0.804662	0.730842	0.005073	0.004876	0.003283	0.006823	0.003565
BoW	LinearSVM	0.876786	0.809010	0.977882	0.794674	0.726078	0.004212	0.004137	0.003405	0.006236	0.003103
W2V-1	LogisticRegression	0.868929	0.793731	0.951530	0.799552	0.692454	0.004362	0.004262	0.002615	0.005917	0.003136
W2V-1	LinearSVM	0.866450	0.790770	0.953871	0.793750	0.691704	0.005874	0.005800	0.002995	0.007568	0.004246

Table 2: Nested CV Results: mean metrics and standard deviations

Model selection In Table 3 has been presented the pairwise comparisons matrix across all model–vectorizer combinations. A value of 1 indicates that the model in the row significantly outperformed the model in the column in head-to-head statistical tests, while a value of -1 indicates that the model underperformed the model in column; whereas 0 denotes no statistically significant win.

	LR + TF-IDF	LR + BoW	LSVM + TF-IDF	RF + TF-IDF	RF + BoW	LSVM + BoW	RF + W2V-1	LR + W2V-1	LSVM + W2V-1
LR + TF-IDF	0	1	1	1	1	1	1	1	1
LR + BoW	-1	0	1	1	1	1	1	1	1
LSVM + TF-IDF	-1	-1	0	1	1	1	1	1	1
RF + TF-IDF	-1	-1	-1	0	1	1	1	1	1
RF + BoW	-1	-1	-1	-1	0	1	1	1	1
LSVM + BoW	-1	-1	-1	-1	-1	0	1	1	1
RF + W2V-1	-1	-1	-1	-1	-1	-1	0	1	1
LR + W2V-1	-1	-1	-1	-1	-1	-1	-1	0	1
LSVM + W2V-1	-1	-1	-1	-1	-1	-1	-1	-1	0

Table 3: Pairwise victory matrix (1 = win, 0 = tie, -1 = loss)

Table 4 summarizes the scores for each pipeline(summation on previous table rows), giving an at-a-glance ranking of overall comparative performance. Higher counts reflect more frequent wins against competing configurations.

Model	**Score**
RandomForest + TF-IDF	8
RandomForest + BoW	6
RandomForest + W2V-1	4
LogisticRegression + TF-IDF	2
LinearSVM + TF-IDF	0
LogisticRegression + BoW	-2
LinearSVM + BoW	-4
LogisticRegression + W2V-1	-6
LinearSVM + W2V-1	-8

Table 4: Number of head-to-head victories per model

Focusing on the top-ranked model, Table 5 details the results of the pairwise statistical tests used to establish significance. For each winner–opponent pair has been reported the test type, test statistic, p-value, and the final outcome (win/no win).

Winner	Opponent	Test	Stat.	p-value	Outcome
RandomForest + TF-IDF					
	RandomForest + BoW	t-test	5.3412	0.0000	win
	RandomForest + W2V-1	t-test	6.9873	0.0000	win
	LogisticRegression + TF-IDF	t-test	52.5093	0.0000	win
	LinearSVM + TF-IDF	t-test	61.5744	0.0000	win
	LogisticRegression + BoW	t-test	46.4346	0.0000	win
	LinearSVM + BoW	t-test	57.6489	0.0000	win
	LogisticRegression + W2V-1	t-test	51.3970	0.0000	win
	LinearSVM + W2V-1	t-test	52.3118	0.0000	win

Table 5: Head-to-head statistical tests for the top model (RandomForest + TF-IDF)

Among all tested configurations, RandomForest combined with TF-IDF achieved the highest F1 score, indicating superior overall in classification performance.

3.2 Multiclass Classification Results

Experimental Setup and Grid Search. As described in Section 2.4.1, three classifiers Random Forest, Logistic Regression, and Linear SVM were evaluated in combination with three vectorization methods: Bag of Words (BoW), Term Frequency-Inverse Document Frequency (TF-IDF), and custom-trained Word2Vec embedding (W2V-1). This led to a total of 9 distinct model configurations. The grid of hyperparameters used during tuning was the following:

- **Logistic Regression:**
  - `C`: {0.01, 0.1, 1}
- **Linear SVM:**
  - `C`: {0.01, 1, 10}
- **Random Forest:**
  - `n_estimators`: {100, 500}
  - `max_depth`: {None, 20}
  - `random_state`: 42

Model evaluation Table 6 reports the optimal hyperparameters that were selected via nested cross-validation for each vectorizer–classifier pipeline in the multiclass setting. For each configuration, we show the mode of the best parameters found across the ten outer folds.

Configuration	Chosen Parameters
BoW + LogisticRegression	`'model__C': 1`
BoW + RandomForest	`'model__max_depth': None, 'model__n_estimators': 500, 'model__random_state': 42`
BoW + LinearSVM	`'model__C': 1`
TF-IDF + LogisticRegression	`'model__C': 1`
TF-IDF + RandomForest	`'model__max_depth': None, 'model__n_estimators': 500, 'model__random_state': 42`
TF-IDF + LinearSVM	`'model__C': 1`
W2V-1 + LogisticRegression	`'model__C': 1`
W2V-1 + RandomForest	`'model__max_depth': None, 'model__n_estimators': 500, 'model__random_state': 42`
W2V-1 + LinearSVM	`'model__C': 1`

Table 6: Chosen parameters for each configuration (mode over 10 folds) – Multiclass

Table 7 then summarizes the key evaluation metrics: mean accuracy, precision, recall and F1-macro—along with their standard deviations across the outer folds. This allows for a comprehensive comparison of both the average performance and the consistency of each model.

Vectorizer	Classifier	Mean F1	Mean Acc.	Mean Prec.	Mean Rec.	Std F1	Std Acc.	Std Prec.	Std Rec.
TF-IDF	RandomForest	0.932961	0.936677	0.933369	0.934142	0.006951	0.006662	0.006662	0.006977
BoW	RandomForest	0.927850	0.931865	0.928208	0.928836	0.004708	0.004557	0.004444	0.004664
BoW	LogisticRegression	0.927084	0.930480	0.928832	0.928977	0.006481	0.006258	0.006051	0.006484
TF-IDF	LinearSVM	0.926727	0.930249	0.928194	0.928430	0.007833	0.007524	0.007287	0.007826
TF-IDF	LogisticRegression	0.926651	0.930085	0.928354	0.928411	0.007372	0.007165	0.006745	0.007355
BoW	LinearSVM	0.925820	0.929228	0.928047	0.927851	0.007079	0.006787	0.006615	0.007181
W2V-1	RandomForest	0.892870	0.896660	0.898444	0.895279	0.005872	0.005615	0.005968	0.005940
W2V-1	LinearSVM	0.878508	0.885090	0.879590	0.879230	0.005022	0.004805	0.004660	0.004996
W2V-1	LogisticRegression	0.878328	0.884299	0.879644	0.878921	0.005467	0.005315	0.005062	0.005452

Table 7: Nested CV Results: mean metrics followed by standard deviations – Multiclass

Model selection Table 8 shows the pairwise comparisons matrix across all model–vectorizer combinations. A value of 1 indicates that the model in the row significantly outperformed the model in the column according to a statistical test, while -1 denotes an underperforming and 0 indicates that there is no statistical difference.

	RF + TF-IDF	RF + BoW	LR + BoW	LSVM + TF-IDF	LR + TF-IDF	LSVM + BoW	RF + W2V-1	LSVM + W2V-1	LR + W2V-1
RF + TF-IDF	0	1	1	1	1	1	1	1	1
RF + BoW	-1	0	0	0	1	1	1	1	1
LR + BoW	-1	0	0	0	1	1	1	1	1
LSVM + TF-IDF	-1	0	0	0	0	0	1	1	1
LR + TF-IDF	-1	-1	-1	0	0	0	1	1	1
LSVM + BoW	-1	-1	-1	0	0	0	1	1	1
RF + W2V-1	-1	-1	-1	-1	-1	-1	0	1	1
LSVM + W2V-1	-1	-1	-1	-1	-1	-1	-1	0	0
LR + W2V-1	-1	-1	-1	-1	-1	-1	-1	0	0

Table 8: Pairwise victory matrix (1 = win, 0 = draw, -1 = loss)

Table 9 reports the score of the pipelines. A higher number indicates stronger overall performance in the statistical comparison.

Model	Score
RandomForest + TF-IDF	8
RandomForest + BoW	4
LogisticRegression + BoW	4
LinearSVM + TF-IDF	2
LogisticRegression + TF-IDF	0
LinearSVM + BoW	0
RandomForest + W2V-1	-4
LinearSVM + W2V-1	-7
LogisticRegression + W2V-1	-7

Table 9: Number of head-to-head victories per model

Table 10 lists the statistical test results for the top-ranked pipeline. For each comparison, the test type, statistic, p-value, and outcome (win or no win) are reported.

Winner	Opponent	Test	Stat.	p-value	Outcome
RandomForest + TF-IDF					
	RandomForest + BoW	Wilcoxon	1.0000	0.0000	win
	LogisticRegression + BoW	t-test	8.4392	0.0000	win
	LinearSVM + TF-IDF	t-test	8.5906	0.0000	win
	LogisticRegression + TF-IDF	t-test	10.6920	0.0000	win
	LinearSVM + BoW	t-test	9.7807	0.0000	win
	RandomForest + W2V-1	t-test	44.3189	0.0000	win
	LinearSVM + W2V-1	t-test	50.9456	0.0000	win
	LogisticRegression + W2V-1	t-test	46.3550	0.0000	win

Table 10: Head-to-head statistical tests for the top model (RandomForest + TF-IDF)

RandomForest + TF-IDF emerged as the pipeline with statistically superior performance so it represents the final choice as the multiclasse classifier to feed into the overall binary-plus-multiclass pipeline. Random Forest offers native global interpretability through its built-in feature-importance scores and seamless local explanations via TreeInterpreter, while TF-IDF delivers a normalized feature space where each term's weight can be assessed directly.

Further Observation From the confusion matrices(it's been taken the confusion matrix due its easy interpretability), it's possible observe that most misclassifications occur between the `gender` and `other_cyberbullying` classes. This is likely due to the semantic overlap between expressions of gender-based harassment and more generic forms of offensive language. Notably, this pattern

of confusion is consistent with observations reported in previous work[3], where misclassifications between gender and other_cyberbullying categories accounted for the majority of classification errors. Nevertheless, class-wise performance remains robust, and no class shows signs of significant bias or neglect.

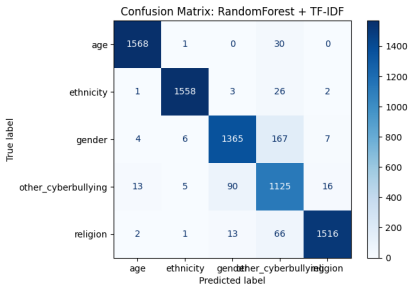


Figure 5: Multiclass confusion matrix.

3.3 Pipeline Evaluation

In this final evaluation, the full classification pipeline was tested in its complete form. The system first applies a binary classifier to distinguish between cyberbullying and non-cyberbullying tweets. The tweets identified as cyberbullying (true positives) are then passed to a second classifier, which assigns a specific type of cyberbullying.

This two-stage structure reflects a realistic moderation workflow, where content is first flagged as problematic and then further analyzed for nature.

3.3.1 Binary Stage Performance.

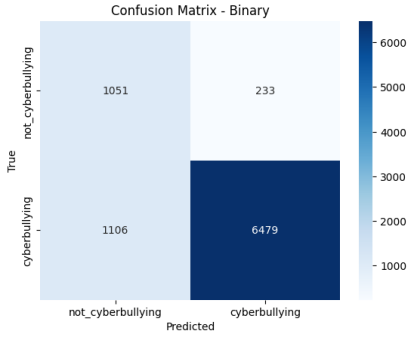


Figure 6: Confusion matrix for binary stage.

[3]Wang, J., Fu, K., & Lu, C.-T. (2020). SOSNet: A Graph Convolutional Network Approach to Fine-Grained Cyberbullying Detection. In their analysis of the confusion matrix, the authors found that gender and other were the most confused classes, accounting for the majority of the overall error.

3.3.2 Multiclass Stage Performance (on true positives).

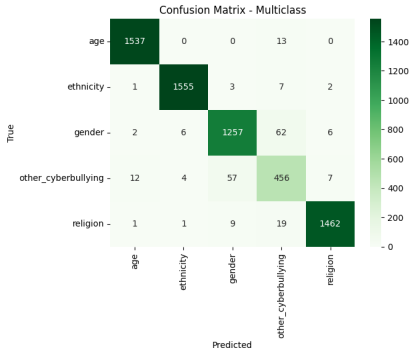


Figure 7: Confusion matrix for multiclass stage (on binary true positives).

Overall Pipeline Performance. By combining the two stages, the overall pipeline correctly classified 7318 tweets out of 8251, resulting in an **end-to-end accuracy of** 0.8251. This metric reflects the proportion of tweets that were both correctly identified as cyberbullying/non-cyberbullying, and, when applicable, correctly categorized into one of the fine-grained classes.

Further Consideration A notable aspect of the evaluation is the improvement in classification accuracy from the multiclass stage alone (0.94) to the same model when used within the pipeline (0.97), despite being applied to the same original test set. This raises an important question: why does the accuracy improve if the dataset is unchanged?

The answer can be found by analyzing the confusion matrix of the multiclass classifier within the pipeline. The `other_cyberbullying` category, which is the most generic and typically the hardest to classify, appears significantly underrepresented compared to the other labels. However, the original dataset is not so unbalanced across classes; suggesting that this discrepancy is not due to the input data.

The cause lies in the binary classifier, which struggles to identify `other_cyberbullying` tweets as harmful. As a result, many examples from this category are incorrectly filtered out in the first stage and never reach the multiclass classifier. This leads to a lower number of `other_cyberbullying` examples in the second stage inflating its accuracy.

This hypothesis is supported by the label distribution shown in the final classification report: only 536 instances of `other_cyberbullying` are processed at the second stage, compared to 1249 istances of the complete test set. Additionally, performance metrics for this class are noticeably lower than for the others, confirming that it remains the most challenging category for the pipeline to handle. So having less occurrences of most problematic labels we can observe an "improvement" of accuracy.

Label	Precision	Recall	F1-score	Support
age	0.99	0.99	0.99	1550
ethnicity	0.99	0.99	0.99	1568
gender	0.95	0.94	0.95	1333
other_cyberbullying	0.82	0.85	0.83	432
religion	0.99	0.98	0.98	1492

Table 11: Per-class metrics for the multiclass classifier (pipeline stage).

3.4 Explainability Insights

To support transparency and user trust, the system integrates both global and local explainability modules.

Global Insights. To gain a high-level understanding of the model's behavior, two complementary techniques were used.

Pattern mining was applied to identify the most frequent and informative terms associated with each cyberbullying class. These word associations provide an intuitive view of how language patterns differ across abuse categories. To better understand the structure of these patterns, the distribution of closed and maximal itemsets across the classes was analyzed.

Class	Number of closed itemsets	Number of maximal itemsets
Multiclass Target		
age	144	24
ethnicity	127	26
religion	68	31
gender	62	17
other_cyberbullying	8	8
Binary Target		
cyberbullying	54	23
not_cyberbullying	4	4

Table 12: Distribution of closed and maximal itemsets across multiclass and binary targets

The analysis shows that the categories `age` and `ethnicity` exhibit the highest number of closed itemsets (144 and 127, respectively), indicating the presence of numerous recurring linguistic patterns. Interestingly, these patterns then condense into a smaller number of maximal itemsets (24 and 26, respectively), suggesting that the variety of observed patterns tends to converge into more representative and non-redundant linguistic structures. This phenomenon reflects a certain lexical richness and consistency in the messages related to these categories.

Focusing on the maximal itemsets, which represent the strongest and most concise associations, `religion` stands out with the highest number (31), followed by `ethnicity` (26), `age` (24), and `gender` (17). This suggests that, al-

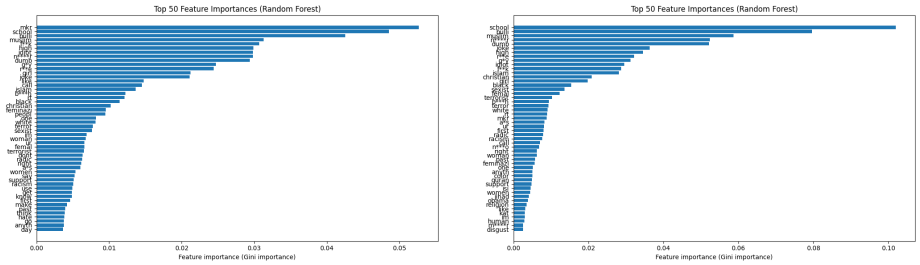
though `age` and `ethnicity` display greater initial variety, content related to `religion` tends to crystallize into more compact and specific patterns, while `gender` occupies an intermediate position between consistency and variability.

In fact, the `gender` category, despite having fewer closed itemsets (68) than `age` and `ethnicity`, still reveals a relatively stable linguistic structure, as shown by its 17 maximal itemsets. This indicates that although the patterns are less varied, some lexical associations are strong enough to emerge as representative.

By contrast, the `other cyberbullying` class displays an extremely limited number of patterns (only 8 itemsets, both closed and maximal), indicating a low recurrence of common linguistic structures, likely due to the greater intrinsic heterogeneity of the class.

Overall, these findings confirm what was observed during the multiclass classification phase: the `other cyberbullying` class is the most difficult to identify, even from a lexical perspective, while categories such as `age`, `ethnicity`, `religion`, and, to a slightly lesser extent, `gender`, appear more clearly identifiable thanks to stronger and more coherent linguistic associations. When shifting the focus to the binary setting, where the goal is to distinguish between cyberbullying and non-cyberbullying texts, a different pattern emerges. The cyberbullying class yields 54 closed itemsets and 23 maximal ones, indicating a high compression rate and limited redundancy. More strikingly, the `not_cyberbullying` class produces only 4 closed itemsets, all of which are also maximal. This perfect overlap highlights an extremely sparse and flat pattern structure, likely due to the lexical heterogeneity and general nature of non-abusive content. In such cases, the absence of recurring co-occurrences prevents the emergence of structured itemsets, making this class inherently less amenable to pattern-based discrimination. As a result, most meaningful structures arise only from the cyberbullying class, where offensive or harmful expressions are more likely to cluster in consistent and repeated forms. This reinforces the idea that pattern mining approaches, particularly those targeting redundancy-aware or closed/maximal itemsets, are more effective when applied to abusive content, while neutral or generic language tends to resist such compression.

In a second instance, for both binary and multiclass classifier, the `feature_importances_` attribute of the Random Forest classifiers trained on TF-IDF features was analyzed. The top 50 most important terms are reported in Figures 8, showing which words had the greatest influence on the model's overall decision-making.



(a) Bynary (b) Multiclass

Figure 8: top 50 most important features based on Random Forest `feature_importances_`.

As can be observed, most of the top-ranked terms in the binary classification model are strongly associated with offensive, aggressive, or otherwise harmful language. This finding corroborates the results obtained through pattern mining, where meaningful and recurrent structures emerged predominantly within the cyberbullying class. It suggests that the model is primarily learning to detect the presence of abusive content, rather than performing a balanced discrimination between cyberbullying and non-cyberbullying instances. This, once again, underlines the fact that harmful messages tend to exhibit more specific, repeated, and structured lexical patterns compared to the diverse and unstructured nature of neutral or non-abusive content. From this perspective, the model's behavior is not only expected, but also aligned with the way humans tend to recognize and respond to toxicity in communication. Indeed, when exposed to a message, a human intuitively perceives whether or not bullying has occurred; they do not approach every tweet by asking themselves whether it qualifies as cyberbullying.

Local Insights. To explain individual predictions produced by the multiclass classifier, the TreeInterpreter tool was employed. This method breaks down the decision of a tree-based model into the contributions of each input feature, allowing for a detailed analysis of how specific terms influenced the classification outcome.

One particularly interesting insight emerging from this analysis concerns the role of feature absence. In addition to quantifying the positive or negative impact of words that appear in the input text, TreeInterpreter also highlights how the absence of certain terms can affect the prediction. In the context of TF-IDF representations, this means that words which typically characterize other classes — but are missing from the current input — can decrease the likelihood of those classes being predicted, effectively reinforcing the assigned label.

For example, consider the tweet:

```
"I think you are a spoiled brat"
```

TreeInterpreter reveals that terms such as `"sploiled"` and `"brat"` made strongly positive contributions toward the prediction of the `age` cyberbullying class.

At the same time, the absence of features typically linked to other classes, such as gendered terms, racial or ethnic identifiers, negatively impacted their respective class probabilities. This absence, combined with the presence of highly indicative features for the `age` class, increased the model's overall confidence in its prediction.

This kind of interpretability not only clarifies why a particular label was assigned, but also uncovers subtle linguistic dynamics that may not be immediately apparent to a human reviewer.

Post-hoc interpretability methods often involve a significant computational burden, which can pose challenges in practical applications. For this reason, in the cited paper, a comparison was made between TreeInterpreter and SHAP TreeExplainer (SHAP-TE), two widely used attribution methods for tree-based models. Although SHAP-TE provides theoretical consistency guarantees, empirical evidence shows that TreeInterpreter offers comparable attribution quality with significantly lower computational cost.[4] Given these considerations, TreeInterpreter was selected as the preferred method for interpreting predictions in our multiclass classification setting.

feature	contribution	tfidf_value
school	0.180766	0.335520
bulli	0.145115	0.283278
g*y	0.065892	0.427410
high	-0.043878	0.000000
girl	-0.034279	0.000000
r**e	-0.024721	0.000000
mkr	0.022816	0.000000
joke	-0.022368	0.000000
realli	0.018964	0.519215
middl	-0.013784	0.000000
like	-0.012949	0.000000
f**k	-0.012708	0.000000
day	0.010788	0.000000
dumb	-0.010145	0.000000

feature	contribution	tfidf_value
dumb	0.209906	0.597026
n****r	0.196146	0.592484
f**k	0.144600	0.540853
mkr	0.018358	0.000000
school	-0.008127	0.000000
high	-0.006126	0.000000
a*s	-0.004954	0.000000
r**e	-0.004876	0.000000
muslim	-0.004699	0.000000
girl	-0.004511	0.000000
g*y	-0.004501	0.000000
idiot	-0.004402	0.000000
joke	-0.003925	0.000000
like	-0.003748	0.000000

feature	contribution	tfidf_value
sexist	0.244969	0.444443
woman	0.156792	0.415828
im	0.090007	0.315214
hate	0.074928	0.404231
rt	0.042706	0.291108
mkr	0.032040	0.000000
comment	0.023050	0.531085
school	-0.022331	0.000000
christian	-0.020823	0.000000
right	-0.015958	0.000000
f**k	-0.015942	0.000000
muslim	-0.013113	0.000000
n****r	-0.012289	0.000000
dumb	-0.011584	0.000000

Figure 9: Local examples explained using TreeInterpreter.

4 Graphical User Interface

The graphical user interface (GUI) of the *Cyberbullying Detection* system is designed to be user-friendly while providing interpretable results. It is implemented using `Tkinter` and consists of two main views: the classification interface and the explanation interface.

[4] P. Sharma et al., "Evaluating Tree Explanation Methods for Anomaly Reasoning: A Case Study of SHAP TreeExplainer and TreeInterpreter," *arXiv preprint* arXiv:2010.06734, 2020.

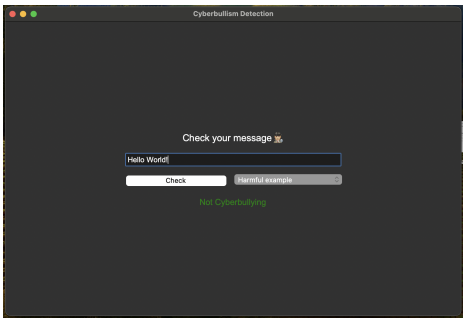


Figure 10: Main interface: the message "Hello World!" is correctly classified as safe.

In the main window (Figure 10), the user is prompted to enter a message into a text field. Upon clicking the `Check` button, the system analyzes the content using a binary classifier trained to distinguish cyberbullying from harmless messages. If the message is safe, the interface displays a green label saying *Not Cyberbullying*. Otherwise, it shows a red warning along with the specific type of cyberbullying detected (e.g., *ethnicity*, *sexual*, etc.), as illustrated in Figure 11.

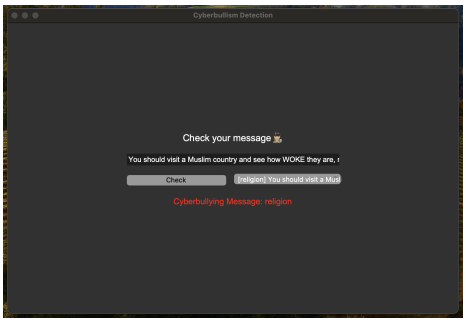


Figure 11: Example of a detected cyberbullying message classified under the 'religion' category.

To enhance interpretability, the application includes an additional explanation panel (Figure 12) that provides insights into the model's decision process. This window is divided into three sections:

- **Left panel – TreeInterpreter analysis:** displays the top 50 textual features (word stems) that contributed to the classification. Each row includes the word, its contribution score (positive or negative), and its TF-IDF value. This breakdown helps understand which terms were most influential in the prediction.
- **Top-right panel – Class word distribution:** a bar chart shows the 25 most frequent words associated with the predicted cyberbullying category.

This gives a visual representation of common vocabulary within the class.

- **Bottom-right panel – Closed/maximal Itemsets:** this section lists the most relevant itemsets mined from the dataset for the predicted class. These itemsets highlight frequent co-occurring patterns of words that characterize each type of cyberbullying. In addition to the itemsets is specified the type, closed or maximal, and the relative support of them.

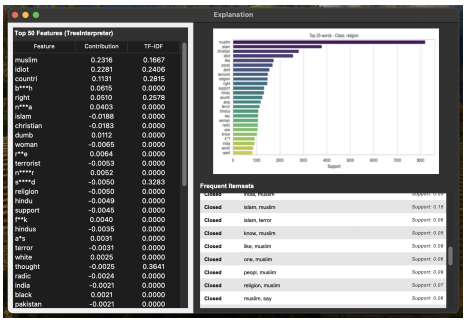


Figure 12: Explanation interface: feature contributions (left), top words by support (top-right), and closed and maximal itemsets (bottom-right).